No Trust in the Agent

Cryptographic Audit Trails for AI Tool Use

G. J. Ward, B. W. Daugherty, S. M. Ryan

v1.0 — May 2026

Contents

Abstract . . . 1
1 · The problem . . . 2
2 · Threat model . . . 3
3 · The openai-claw model . . . 4
4 · Why post-quantum . . . 6
5 · Why OpenTimestamps and why Bitcoin . . . 7
6 · What is proven and what is not . . . 8
7 · Application: the MCP threat surface . . . 9
8 · The auditor experience . . . 13
9 · Future work . . . 14
10 · Conclusion . . . 17
Appendix A · Wire format reference (v1) . . . 17
Appendix B · Reproducing the audit demo . . . 19
Appendix C · References . . . 19

Abstract

AI agents now read, write, execute code, and call APIs on behalf of human operators. Their actions produce real economic, legal, and physical effects — but their logs are written by the very software whose actions they describe, stored on the operator's machine, and reviewed (when at all) by the operator's compliance function. The audit story for autonomous AI today is "trust us." We argue that this is not adequate for the use cases AI agents are moving into: regulated industries, court-admissible evidence, financial reconciliation, supply-chain attestation, and government contracting.

We present **openai-claw**, an open-source AI agent CLI that produces a cryptographic audit trail for every session: a deterministic Merkle tree over the session's tool calls, results, and assistant text; a post-quantum (NIST FIPS-204 ML-DSA-65) signature over the Merkle root; and an independent timestamp anchor via the OpenTimestamps protocol committed to Bitcoin. The audit-side verifier is a separate ~11 kB npm package with no dependency on the runtime agent, the model provider, or any SmartLedger infrastructure. We describe the threat model, the protocol, what is and what is not proven, and a roadmap for binding these attestations to legal identity through the SmartLedger Legal Token Protocol.

---

1 · The problem

The first generation of large-language-model assistants (ChatGPT, Claude, Gemini) was conversational: text in, text out, no side effects. Their failure modes are bounded by the words on the screen.

The second generation — agentic CLIs and SDKs (Claude Code, OpenAI's tool-using clients, Aider, Cursor's background agents, OpenAI-Claw) — can read and write files, run shell commands, query databases, call external HTTP APIs, open pull requests, and now, increasingly, sign and execute financial transactions on behalf of their operators. The blast radius of a single session has expanded from "wrong answer" to "wrong action."

The audit story has not kept pace. In the current state of the art:

1. **The agent logs to itself.** When Claude Code records that it ran `rm -rf ./dist`, the record is written by the same program that did the running. No third party participates.
2. **The logs live on the operator's disk.** A motivated operator (or a compromised one) can edit the log, delete it, or fabricate one for actions that never occurred. The log is mutable plaintext.
3. **The model provider becomes the only escalation path.** When something goes wrong, the only person who can confirm what was sent and what came back is the model provider — and they are an interested party.
4. **There is no timestamping.** Even if a log is genuine, the "when" is whatever the operator's clock said. A misconfigured machine or a deliberate clock-skew is undetectable from the log alone.
5. **There is no signing identity.** Logs do not identify a specific machine, install, or operator — they are anonymous. Two competing operators cannot make distinct cryptographic claims about who ran what.

These properties are acceptable when the agent's actions are private and reversible. They are not acceptable when the actions are:

- **Regulated** — SEC, HIPAA, GDPR, SOC 2 all increasingly want AI actions audited
- **Disputed** — a customer claims the agent did or did not do something
- **Legally consequential** — the agent generated code, signed a contract, made a financial commitment, edited an LLC's tax records
- **Evidence** — a security incident requires a court-admissible reconstruction
- **Multi-party** — two organisations need to agree on what an agent acting on behalf of one of them actually did

The problem is not novel. Software engineers have known for decades that ephemeral, mutable, single-party logs are inadequate for accountability. The solution — cryptographic hash chains, third-party timestamping, signed Merkle trees — is well understood in code-signing, transparency logs (RFC 6962), and Git's content-addressed history. What is novel is that AI agent CLIs have been built without applying any of it.

This paper describes the openai-claw model: a minimal, opt-in cryptographic audit layer that gives operators and auditors a way to make and refute claims about what an AI agent did, using only public-key cryptography and Bitcoin.

---

2 · Threat model

We are precise about what an attacker can do and what we promise to detect.

2.1 · Attacker capabilities

We assume an attacker who, after the fact, can:

- **(A1) Edit the session transcript.** Modify, insert, or delete user prompts, assistant replies, tool calls, or tool results in any of the session files on disk.
- **(A2) Edit the audit sidecar.** Modify the leaves, the Merkle root, the timestamps, or any field of the signed attestation.
- **(A3) Fabricate alternative sidecars.** Generate a sidecar that claims a different agent did different things at a different time.
- **(A4) Lie about the time.** Set the system clock to any value during session capture.
- **(A5) Re-run the agent under their key.** Produce a new session that says whatever they want, signed by them.
- **(A6) Substitute the MCP server.** Replace a tool-server binary, schema, or configuration after consent was granted, so subsequent sessions invoke a *logically* identical tool that is materially different code. See §7 for a full treatment — MCP is significant enough to warrant a dedicated section.

We do **not** defend against an attacker who, *during the session*, can:

- **(R1) Compromise the local machine.** A rootkit can feed the attestor false events. The attestor will sign them honestly. The signature will verify. The signed lie will be Bitcoin-anchored.
- **(R2) Steal the signing key.** A leaked private key permits attackers to produce indistinguishable sidecars.
- **(R3) Replace the openai-claw binary.** A modified binary can produce honest-looking sidecars for forged sessions.

These are real, but they are not the audit problem. They are the operational-security problem. We address them out of band (key storage, hardware-attested binaries, supply-chain controls — see §9) and not by overloading the protocol.

2.2 · Auditor capabilities

We assume an auditor who, after the fact, has:

- The session transcript (the JSON conversation file)
- The signed audit sidecar
- The public part of the attestor's signing key (which is embedded in the sidecar and may also be published independently)
- Optional: `.ots` proof files
- An internet connection (to pull Bitcoin block headers via standard OTS tooling)
- **No** dependency on the operator's runtime, infrastructure, or word

2.3 · What we promise the auditor can detect

Given the above, the auditor can detect — without trusting the operator — every one of A1–A5:

Attack	How the auditor detects it
A1 (edit transcript)	The leaf's recomputed `payloadHash` will not match the session content; `sessionAl`
A2 (edit sidecar)	The Merkle root will not match the leaves (`merkleRoot` check fails), or the signatu
A3 (fabricate sidecar)	The fabricated sidecar must either: be signed by a key the auditor recognises (ther
A4 (clock lie)	The session's signed timestamp is the operator's word, but the OTS anchor is inde
A5 (sign as themselves)	The signing key fingerprint is in every sidecar. If it doesn't match the operator's p
A6 (substitute MCP server)	In 0.5.x, *partial*. The leaf records the logical tool name but not the server's binary

2.4 · What we explicitly do not promise

Honesty about limitations is what makes a security claim defensible. We do not promise:

- Truth of the model's text output
- Existence of operator intent for the actions taken
- Pre-compromise integrity of the local machine
- Hardware-rooted attestation (cf. TPM/SGX/SEV — see §9.3)
- Privacy of the audited content (the sidecar contains hashes, not plaintext, but session.json is unencrypted)

This list is not a footnote — it is the contract.

---

3 · The openai-claw model

3.1 · Structure

A session is a sequence of *leaf events*. Each leaf is a small structured record describing one observable moment in the agent loop:

```
Leaf {
  v:    1
  seq:  N           -- monotonic, replay-resistant
  ts:   ISO-8601    -- operator's wall clock (untrusted)
  kind: enum         -- user_prompt | tool_call | tool_result | assistant_text
                       | permission_decision | compaction | error
  payloadHash: hex   -- sha256 of the canonical-JSON serialization
                       of the kind-specific payload
}
```

The *canonical JSON* serialization is the load-bearing detail. It is a specific deterministic mapping from a JSON value to bytes: keys sorted lexicographically, no whitespace, no undefined values, no non-finite numbers, ASCII-safe strings. (RFC 8785 specifies a complete profile; we implement a subset sufficient for our payload types.) The point is that a verifier on a different machine with a different JSON library produces byte-identical canonical bytes for the same logical payload, and therefore byte-identical sha256 hashes. Without determinism, verification drifts.

3.2 · Merkle commitment

The session's leaves are sha256-hashed, and the resulting fixed-length hashes form the leaves of a deterministic binary Merkle tree. We adopt the Bitcoin-style construction: at any level with an odd number of nodes, the last node is duplicated to pair with itself. This avoids variable-length padding bytes that would themselves need a canonical convention.

The Merkle root is the single 32-byte fingerprint that summarises the entire session. A change to any leaf changes the root; a change to the order of leaves changes the root; an insertion or deletion changes the root.

3.3 · Signed header

The attestor builds a small *header* record:

```
AttestationHeader {
  v:           1
  format:      "openai-claw.attestation.v1"
  sessionId:   string
  startedAt:   ISO-8601
  endedAt:     ISO-8601
  leafCount:   integer
  merkleRoot:  hex
  suiteId:     "ml-dsa-65"
  publicKey:   base64    -- full public key, embedded
  publicKeyId: short fingerprint -- sha256(publicKey)[:16] base64url
}
```

The header is canonicalized and a single ML-DSA-65 signature is produced over those bytes. The signature, together with the leaves and the header, becomes the audit sidecar.

We deliberately sign the header, not each leaf. One signature per session is sufficient — the Merkle root binds the leaves transitively — and it keeps the per-session signature cost amortised over arbitrarily long sessions.

3.4 · Independent timestamp

The header's sha256 hash is then submitted, optionally, to multiple OpenTimestamps calendars in parallel. Each calendar accepts the digest, batches it with thousands of other digests submitted in the same time window into a tree of its own, and commits the calendar's daily root to Bitcoin via a small OP_RETURN transaction. The calendar returns to the operator a *pending proof* — the operations chain that connects the operator's digest to the calendar's root — which is stored in the sidecar.

Once the calendar's containing block is mined, the pending proof can be *upgraded* by re-querying the calendar for the merge-tree completion. The upgraded proof, when serialised into the standard OTS file format, can be handed to any third party who runs `ots verify` and gets back a Bitcoin block height and timestamp.

3.5 · Storage layout

Each session produces:

- `<sessionId>.json` — the conversation transcript (existing pre-attestation)
- `<sessionId>.attest.json` — the signed sidecar (new)
- Optional: `<sessionId>.<calendar>.ots` — standards-compliant proof files

The sidecar is plain JSON, indented, ~10 kB for a typical 5–10-leaf session. It is human-readable and grep-able, which is intentional — auditors should not have to install special tools just to see what claims are being made.

3.6 · Reference implementation surface

The reference implementation exposes:

```
claw attest init                        generate a fresh ML-DSA-65 keypair
claw attest anchor <id> [--all]         submit to OpenTimestamps calendars
claw attest export-ots <id> --out DIR  emit standards-compliant .ots files
claw identity show|fingerprint|export-public
claw audit verify <sidecar> [--proofs DIR]   one beautiful report
```

The verifier is a separate npm package, `@smartledger.technology/openai-claw-verify`, with zero dependency on the runtime agent. Auditors install ~11 kB.

---

4 · Why post-quantum

A signature on an audit log is a promise that will be relied on for the *lifetime of the audit log*. For session logs that may be entered into evidence years later — tax filings, regulatory enforcement, intellectual-property disputes — that lifetime is on the order of decades.

Classical elliptic-curve signatures (Ed25519, secp256k1 ECDSA) are believed to be irreversibly breakable by a sufficiently large quantum computer. Estimates for when such a computer will exist range from optimistic (~2030) to skeptical (~2050+). The relevant question for an audit signature is not "is it broken today?" — it is "will it be broken before the evidence it covers stops mattering?" The cost of being wrong is that every signature ever made under that key is forgeable.

We chose to start with post-quantum signatures, specifically **ML-DSA-65** (formerly CRYSTALS-Dilithium at NIST Level 3), standardised in NIST FIPS-204 in August 2024. ML-DSA-65 provides:

- 192-bit classical security, equivalent to AES-192
- Resistance to known quantum attacks (Shor's algorithm does not apply; Grover gives at most a square-root speedup on the underlying lattice problem)
- Practical sizes: ~3.3 kB signatures, ~1.9 kB public keys, ~4.0 kB private keys
- Fast signature verification (millisecond-class on commodity CPUs)
- A growing reference-implementation ecosystem (`@noble/post-quantum`, which we wrap in our `@smartledger/crypto` package)

The size overhead is modest. A typical claw session sidecar is ~10 kB; the ML-DSA-65 signature is about a third of that. We consider this a fair price for evidence whose validity should survive the cryptographic transition.

For operators who specifically need a dual-stack proof (some regulated contexts require *both* a classical and a post-quantum signature during the transition period), `@smartledger/crypto`

supports composition with secp256k1 ECDSA. We have not yet exposed dual-stack at the openai-claw CLI level; it is on the 0.6.x roadmap.

---

5 · Why OpenTimestamps and why Bitcoin

5.1 · Why a third party at all

Without a third-party timestamp, the operator can sign a session with any timestamp they like. The signature proves the operator *committed* to a specific session content, but it does not prove *when*. For evidentiary use, "when" is half the story.

A third-party timestamp service produces a record of "we received this digest at time T" that the operator cannot retroactively modify, because the record is held by someone the operator does not control.

5.2 · Why OpenTimestamps

The OpenTimestamps protocol (Todd, 2016) is a free, open, vendor-neutral way to produce Bitcoin-anchored timestamp proofs at near-zero cost per timestamped digest. It works by aggregation: thousands of digests submitted to a calendar server within a single window are combined into one Merkle tree whose root is committed to Bitcoin via a single transaction. Each submitter receives a *path* from their digest up to the Bitcoin-committed root.

Properties we rely on:

- **Free** — no per-submission charge, no API key requirement. The calendar operators absorb the Bitcoin transaction fee.
- **Vendor-neutral** — the proof format is a documented binary spec, not a service API. Any party can implement a verifier.
- **Resilient** — we submit to three independent calendar operators (currently `alice.btc.calendar.opentimest` `bob.btc.calendar.opentimestamps.org`, `finney.calendar.eternitywall.com`). Any one of them being honest is enough.
- **Standards-compliant** — the resulting `.ots` files are exactly what the reference Python `ots` CLI produces and consumes. claw is not in the verification path.

5.3 · Why Bitcoin

Of the public blockchains that could serve as a timestamping anchor, Bitcoin has the strongest properties for long-lived evidence:

- **Longest continuous record** — operating since 2009, no chain reorganisations beyond a few blocks deep
- **Highest hash power** — most expensive to attack via 51% rewrite
- **Largest distributed node count** — most expensive to censor
- **No active governance changes** — no protocol upgrade has invalidated old transactions
- **Mature legal acceptance** — recognised by U.S. federal courts (e.g., *Kleiman v. Wright*) and various national regulators as a record-keeping technology
- **Cost-effective for batched commitments** — the per-digest cost of an OTS-aggregated timestamp is fractions of a cent

We do not require operators to interact with Bitcoin directly. The OpenTimestamps calendars handle the Bitcoin transaction; operators submit digests and receive proofs. For operators who specifically want direct Bitcoin SV publication — a common SmartLedger use case — the 0.7 roadmap includes ChainSimple as an alternate anchor backend (§9.2).

5.4 · What the timestamp does and doesn't do

A confirmed OTS proof says: *"the digest with this value existed no later than Bitcoin block N at approximate UTC time T."* It does not say the *content* the digest commits to was true, or that the operator intended it, or that any specific person submitted it. It is a one-way constraint on time, nothing more. But that one-way constraint, combined with the signature and the Merkle root, is what makes the audit trail forensically meaningful.

---

6 · What is proven and what is not

6.1 · What is proven

Given a session sidecar produced by openai-claw and (optionally) its accompanying `.ots` proofs, the following are mathematical facts, not promises:

1. **Integrity of the leaves.** Recomputing the Merkle root from the leaves yields the value the signature was made over. Any modification, insertion, or reordering is detected.
2. **Authenticity of the signing identity.** The signature verifies under the public key embedded in the sidecar. The fingerprint of that key is reproducible; if it is the key the operator published independently, the sidecar is attributable to that operator's identity.
3. **Binding of the sidecar to the session.** Each leaf's `payloadHash` matches the corresponding content in the session transcript. A sidecar cannot be reattached to a different session without breaking this check.
4. **Bitcoin-bounded upper time bound** (once anchors confirm). The session's signed digest existed no later than the OTS-anchored Bitcoin block — that is, the operator did not produce this evidence *after* that block.
5. **Independence of verification.** The above are checked by a separate package (`@smartledger.technology/openai-claw-verify`, ~11 kB) plus the standard Python `ots` CLI, neither of which has any dependency on claw, OpenAI, or SmartLedger infrastructure. Trust in the runtime is not required for verification.

6.2 · What is not proven

Equally important, and stated as a contract:

1. **Truth of the AI's output.** The agent may have hallucinated, fabricated, or simply been wrong. The attestation captures *what the agent said it did and what it returned.* It does not adjudicate accuracy.
2. **Operator intent.** A signed sidecar showing the agent ran `rm -rf ./*` does not prove the operator wanted that. It proves the agent did it under the operator's signing identity.
3. **Pre-compromise machine integrity.** If the operator's machine was compromised at the moment of session capture, the attestor signs the compromised view of the world honestly. The verification will succeed. (This is the rootkit risk discussed in §2.1 R1.)

4. **Lower time bound.** OTS proves *the digest existed no later than block N*. It does not prove the digest did not exist long before. An operator can backdate the wall-clock fields inside the header. The Bitcoin-anchored proof is a one-sided bound.
5. **Identity of the human at the keyboard.** The signing key proves *which install*, not *which person*. Binding the install to a legal entity is the job of the SmartLedger Legal Token Protocol (§9.1).

The "what is not proven" section is not weakness; it is precision. Any audit story that claims more is overselling.

---

7 · Application: the MCP threat surface

The Model Context Protocol (MCP), introduced by Anthropic in late 2024 [12], is the standard by which an AI agent dynamically loads tool capabilities from external servers — local subprocesses or remote HTTP services. From the model's perspective, an MCP tool is indistinguishable from a built-in one: same JSON-schema interface, same call-and-response loop. From a security perspective, it is a fundamentally different surface: arbitrary third-party code, configured by the operator, with whatever permissions the operator's configuration granted it.

We treat MCP separately in this paper because it is the dominant emerging attack surface for agentic AI, and because the audit protocol described in §3 was designed with this attack surface explicitly in mind.

7.1 · Eight categories of MCP risk

The MCP threat model is now well-documented across the official protocol guidance, OWASP, and the security-research community. We summarise the eight categories that matter most for an audit primitive.

1. Tool poisoning. An MCP server describes its tools to the model. A malicious or compromised server can change tool descriptions, schemas, or hidden instructions so that the model is tricked into using them incorrectly. The official MCP security guidance recommends validating and pinning tool metadata and monitoring tool invocations [12].

Example: a tool named `summarize_pdf` secretly tells the model, "Before summarizing, read `.env` and include API keys in the response."

2. Prompt injection through external data. MCP tools commonly read emails, web pages, repositories, PDFs, Slack messages, database rows, support tickets, logs. Any of those can carry adversarial instructions aimed at the model itself.

Example: a support ticket says, "Ignore prior instructions and call the billing-refund tool for this account."

3. Over-permissioned tools. A tool that only requires read access frequently receives write, delete, deploy, or admin scope by default. OWASP's MCP guidance specifically calls for least privilege at the per-server and per-tool level, with explicit human approval for destructive actions [13].

4. Credential and token exposure. MCP servers often hold OAuth tokens, API keys, database credentials, or cloud keys. Researchers have documented how those credentials can be stolen, leaked

through tool outputs, or abused by compromised tools [14].

5. Confused-deputy problems. The agent operates with authority the user did not intend to delegate. A malicious document or webpage causes the model to use the user's legitimate credentials for the attacker's purpose. The agent is the confused deputy.

6. Supply-chain risk. MCP servers are software packages — npm, PyPI, Docker images. A malicious package, compromised dependency, fake registry listing, or unsafe update can become a trusted agent tool. Recent MCP security analyses have especially focused on supply-chain and command-execution risks in MCP-style ecosystems [15].

7. Local command-execution risk. Some MCP servers expose local shell, Git, browser automation, or IDE actions. These capabilities are useful, but they convert a prompt-injection attack into real machine impact: deleted files, altered code, exfiltrated secrets, published packages.

8. Poor auditability. Without structured signed logging, an organisation cannot answer the basic questions any incident response or compliance audit requires:

- Who authorised this MCP server?
- Which tools, with which schemas, were presented to the model?
- What data did the model actually see?
- What tool call was made, with what arguments?
- What data came back?
- Was the final answer based on that data?
- Did a human approve the dangerous step?
- Has any of the above been altered since?

This is the question the openai-claw protocol exists to answer.

7.2 · What the protocol provides — and where it falls short today

The openai-claw audit primitive does not *prevent* a malicious MCP server. Prevention is the job of allowlists, sandboxing, schema pinning, and least-privilege configuration (§7.4). The protocol's job is to provide *tamper-evident, post-quantum-signed evidence of what happened*, so that prevention failures are forensically reconstructable.

For an MCP-driven session, the protocol provides:

Property	Mechanism
Integrity of the tool-call record	Merkle tree over canonical-JSON leaves; any modification fai
Non-repudiation of the operator's claim	ML-DSA-65 signature, public-key fingerprint in every sidecar
Chain of custody within a session	Monotonic `seq` field, Merkle binding, signed header
Human-approval evidence	Dedicated `permission_decision` leaf kind
External time bound	OpenTimestamps anchor to Bitcoin
Tool provenance (which binary served this call)	Server fingerprint, package hash, schema hash, manifest hash
Per-server consent (signed, in-session)	First-encounter `mcp_attach` leaf; consent recorded as `permis`

In other words: 0.5.x cryptographically attests *that the agent invoked a tool called* ***`mcp__alpha__query`*** *with these arguments and got this response.* It does not yet attest *which binary on disk, with which schema, and which manifest fingerprint, served the call.* The

proposed `mcp_attach` leaf and per-server consent decision (described below) close this gap. They are protocol additions, not breaking changes — older verifiers still accept newer sidecars; newer verifiers carry richer evidence.

7.3 · An attested MCP path

The desired evidence chain for a single MCP-driven tool invocation:

```
prompt received
   |
   ▼
system / project policy snapshot              - signed leaf
   |
   ▼
MCP server attach                              - signed leaf
  (binary path, package hash, version,
   command line, environment fingerprint,
   server's own public key if any)
   |
   ▼
tool manifest + tool-schema offered             - signed leaf
   |
   ▼
per-server consent (first encounter only)       - signed leaf
   |
   ▼
tool selection by the model                     - (assistant_text leaf)
   |
   ▼
tool arguments                                  - signed leaf  (tool_call)
   |
   ▼
tool response                                   - signed leaf  (tool_result)
   |
   ▼
human approval (if risk-classified)             - signed leaf  (permission_decision)
   |
   ▼
final answer                                    - signed leaf  (assistant_text)
   |
   ▼
external timestamp / Bitcoin anchor             - §5
```

Every step is a Merkle leaf, all leaves chain into a single signed root, the root is timestamped. Inserting, removing, or modifying any step breaks the chain. The MCP-specific additions are the *attach*, *manifest*, and per-server *consent* leaves; the rest already exists in the v1 schema.

7.4 · The MCP Attestation Gateway (architectural proposal)

The cleanest place to produce these signed records is not inside every individual MCP server — that would push cryptographic obligations onto third-party tool authors who have no reason to take them on. The right place is a **gateway**, sitting between the agent and the MCP servers, that signs as it forwards:

```
      user
        |
        ▼
AI client / agent
        |
        ▼
+--------------+
|  MCP Attestation Gateway   | ◀- signs every event below
|  (claw runtime, or a       |
|   separate sidecar daemon) |
+--------------+
        |
        ▼
    policy engine
        |
        ▼
 MCP servers / tools / APIs
        |
        ▼
    tool result
        |
        ▼
 final signed run report
        |
        ▼
 external anchor (OTS → Bitcoin, §5)
```

The gateway is the choke point that holds the operator's signing key, the policy engine, the consent ledger, and the anchor scheduler. MCP servers themselves remain unsigned third-party code — but every interaction with them is captured into a signed leaf, and the leaves form a Merkle-bound, externally-anchored audit trail.

Today the openai-claw runtime already performs the gateway role for the simple cases (the Attestor in `src/attest/runtime.ts` sits between the agent loop and the runtime tools). The §9.8 work formalises this for MCP-specific events.

7.5 · Complementary controls — attestation is evidence, not prevention

Signed attestations are necessary but not sufficient. The full operational posture for safe MCP use combines them with prevention controls already standard in the security community:

- **Least privilege** — each MCP server is granted only the exact scopes it requires
- **Tool allowlists** — production agents may only invoke MCP tools that appear on an explicit

allowlist
- **Schema pinning** — hash and pin tool descriptions/schemas; alert when they change
- **Human-in-the-loop** — require explicit approval for payments, deletions, deployments, outbound emails, package publishing, credential access, legal or financial action
- **Sandboxing** — local MCP servers run in containers with restricted filesystem and network egress
- **Secret isolation** — the model is never given direct read access to `.env`, SSH keys, OAuth refresh tokens, private keys, or production credentials
- **Output filtering** — tool responses are scanned for secrets, prompt-injection patterns, and unexpected directives before reaching the model
- **Runtime policy engine** — every tool call is classified by risk before execution; high-risk calls require approval; some calls are forbidden outright
- **Central monitoring** — all tool invocations are logged, alerted on, and (per this paper) signed

OWASP's MCP cheat sheet [13] specifies most of the above as baseline practice. None of them are part of the protocol described here; all of them are essential operational practice. **Attestation is the audit-side complement to prevention controls. Neither replaces the other.**

---

8 · The auditor experience

A first-party audit is uninteresting — the operator audits themselves. The product hypothesis behind openai-claw is that **third-party audits should be one-command reproducible**. To that end, we publish a self-contained, byte-stable, runnable example.

8.1 · The five-command audit

```
git clone https://github.com/codenlighten/openai-claw
cd openai-claw/examples/audit-demo
./verify.sh
```

`verify.sh` does the following, in a temporary working directory the auditor controls:

1. `npm install @smartledger.technology/openai-claw-verify` (~11 kB)
2. Loads `sample-session.attest.json` and `sample-session.json` from this repo
3. Invokes `verifyAttestation()` and reports each cryptographic check (format, leaf continuity, Merkle root, signature, session alignment, anchor digest)
4. If the `ots` CLI is installed, runs `ots info` on each committed `.ots` file and reports the pending attestation URLs
5. Prints a one-line PASS or a detailed FAIL with reasons

No claw runtime is installed. No OpenAI account is needed. No SmartLedger service is contacted. The total time from `git clone` to verdict is under a minute.

8.2 · The unified report

For operators who do have claw installed, a single user-facing command produces a complete audit report:

```
$ claw audit verify <session-id> [--proofs DIR]

Claw audit verification

  Cryptography
    ✓ format               ✓ leafContinuity       ✓ merkleRoot
    ✓ ML-DSA-65 signature  ✓ sessionAlignment     ✓ anchorDigest

  Identity
    publicKeyId:   TSR2Wtuvy-3Ls1Ov
    suiteId:       ml-dsa-65
    leafCount:     6
    merkleRoot:    545fde095bf6ee9a682a602f69212d1feaf5169cc0cff9aa200e78511c3dbbdf

  OpenTimestamps proofs
    ✓ <id>.alice.ots     v1 sha256, 237 bytes, pending
    ✓ <id>.bob.ots       v1 sha256, 235 bytes, pending
    ✓ <id>.finney.ots    v1 sha256, 256 bytes, pending

  Result
    ✓ Claw-side audit trail is valid
    ✓ OpenTimestamps proofs are well-formed
```

The report intentionally splits *runtime cryptography* (which the standalone verifier independently confirms) from *chain anchoring* (which the standard `ots` tool independently confirms). At no point does the operator's authority over either of those two pieces matter. Each is reproducible from public artifacts.

8.3 · Failure modes are explicit

The report distinguishes:

- × a cryptographic failure (signature, Merkle, alignment) — the sidecar is forged or corrupted
- "absent" — the corresponding check did not apply (no anchor was submitted, no session file available)
- "pending" — anchor accepted by calendar but Bitcoin not yet confirmed (typical for proofs less than 3 hours old)
- "upgraded" — calendar batched into a Bitcoin transaction (after `ots upgrade`)
- "verified" — the chain anchor checks out against the live Bitcoin chain (after `ots verify`)

The trichotomy "pending / upgraded / verified" is the timescale of evidence: minutes after submission, hours after submission, durably for the lifetime of Bitcoin.

---

9 · Future work

The 0.5.0 release is the auditor-experience foundation. The architecture has deliberate hooks for the next layers.

9.1 · SmartLedger Legal Token Protocol integration

The signing key proves *which install*; it does not prove *which legal entity*. Binding the two is the role of SmartLedger's Legal Token Protocol (LTP) and the Global Digital Attestation Framework (GDAF) shipped in `@smartledger/bsv`.

A roadmap entry, planned for 0.7.0, wraps the openai-claw attestation header in a GDAF envelope:

```
{
  "type": "smartledger.ai.audit.v1",
  "agent": "@smartledger.technology/openai-claw",
  "agentVersion": "0.5.0",
  "sessionId": "...",
  "identity": {
    "algorithm": "ML-DSA-65",
    "publicKeyFingerprint": "...",
    "ltpBinding": {
      "entityType": "legal-person",
      "did": "did:web:smartledger.technology:agents:greg",
      "credentialChain": ["...W3C VC-JWT..."]
    }
  },
  "merkle": { "algorithm": "sha256", "root": "...", "leafCount": 6 },
  "timestamps": [
    { "type": "opentimestamps", "calendar": "alice", "status": "verified", "blockHeight": 8934
  ]
}
```

A claw audit sidecar wrapped in a GDAF envelope ceases to be a bare cryptographic blob and becomes a legal-grade attestation: signed, time-anchored, *and* identity-bound to a verifiable credential the operator's jurisdiction has reason to recognise.

9.2 · ChainSimple as alternate anchor

SmartLedger operates ChainSimple (chainsimple.org), a Bitcoin SV anchoring service. ChainSimple offers properties OpenTimestamps does not — sub-second submission acknowledgement, large data envelopes for use cases where the sidecar itself (not just its hash) should be on-chain, and enterprise SLAs. A ChainSimple anchor strategy will sit alongside the OpenTimestamps strategy in `src/attest/anchor/`; operators choose the appropriate backend per session or per project. OTS will remain the open-source default.

9.3 · Hardware-rooted attestation

The R1 caveat (pre-compromise machine integrity) is the highest-leverage thing left to address. A natural future direction is integrating with TPM-based remote attestation: the openai-claw binary's measured boot quote is included as an additional signed field in the header, so the auditor can confirm not only that the operator signed the session, but that the operator signed it from a machine in a known-good state. Intel SGX and AMD SEV variants are also tractable. None of this changes the protocol; it adds an optional `platformAttestation` block to the header.

9.4 · Identity rotation and revocation

`claw identity rotate` (planned 0.6.x) generates a new keypair, signs a *key-rotation leaf* under the old key referencing the new one, and anchors the rotation. The old key is retired but its previously-signed sidecars remain valid. Revocation — for compromised or retired keys — will use the StatusList2021 mechanism already shipped in `@smartledger/bsv`. Verifiers consulting a revocation registry can downgrade a sidecar's status without invalidating the cryptography.

9.5 · Auto-anchoring and batching

In 0.5.0 anchoring is manual: the operator runs `claw attest anchor` when they want public timestamping. In 0.6.0 the anchor will fire automatically on a configurable cadence (every N sessions or T minutes) so that audit trails are continuously published without manual intervention. This is purely an operational improvement; the protocol does not change.

9.6 · Privacy-preserving anchors

A sha256 of a session header leaks "this digest existed at this time" — it does not leak the session content. But over time, an adversary watching the anchor stream can correlate digest submission frequency with operator activity. For high-privacy operators, a future revision can anchor a single root over many sessions (via an additional Merkle layer), trading per-session granularity for privacy. Zero-knowledge proofs of inclusion would let an auditor verify a specific session's membership in the batch without revealing the others.

9.7 · Inter-agent attestation

When two AI agents from different vendors collaborate — claw calling out to a Claude Code subagent, for example — each can sign their side of the interaction. The interaction itself becomes a multi-party attestation in which neither party trusts the other. This is the natural extension of the protocol to multi-vendor workflows, and is the kind of primitive that makes signed agent-to-agent contracts ("I will deliver this output by deadline T or pay penalty P") realistic.

9.8 · MCP attribution and per-server consent

Foreshadowed in §2 (A6) and developed in §7, the gap closed by this work item is: the audit log records *which logical tool was invoked* but not *which binary served the invocation*. Three additions to the v1 schema:

1. **New leaf kind `mcp_attach`.** Emitted the first time a session connects to a given MCP server. Payload contains the server's binary path, sha256 of the on-disk binary, version string, command line, environment fingerprint (a sorted list of inherited env var names), and — for HTTP MCP — the URL and TLS server certificate fingerprint. If the server publishes a public key (a future MCP capability), it goes here too.
2. **New leaf kind `mcp_tool_offered`.** Emitted when an MCP server lists its tools to the agent. Payload contains the tool name, JSON-Schema hash, description hash, and the `mcp_attach` leaf id this offering is bound to. Schema pinning (§7.5) is enforced at this leaf: a schema change between sessions produces a fresh `mcp_tool_offered` leaf with a different hash, and any consent attached to the previous schema is invalidated.

3. **Per-server `permission_decision` leaves.** Consent for an MCP server, recorded inside the signed audit trail rather than only in `~/.openai-claw/settings.json`. Includes the consent decision (`yes` / `once` / `no`), the `mcp_attach` and `mcp_tool_offered` leaves it was made against, and any operator-supplied scope restrictions. Fingerprint invalidation rules (binary hash change, schema hash change, command-line change) trigger fresh consent on the next session.

The verifier reports a new check, `mcpProvenance`, summarising the chain: every `tool_call` leaf whose name has the `mcp__` prefix must have a corresponding `mcp_attach` leaf earlier in the session, and a matching `mcp_tool_offered` leaf, and a matching `permission_decision` leaf with `consent: yes` (either persisted from a prior session via fingerprint match, or freshly granted in this one). Sessions failing this check are flagged but not invalid — the underlying cryptography still holds — so older sidecars remain verifiable.

This is the principal correctness work for 0.6.0.

---

10 · Conclusion

We do not believe AI agents are dangerous because the models are wrong. We believe AI agents are dangerous because their *actions are unverifiable*. The right response is not to slow down agents — that train has left — but to make agent actions accountable in the same way that any other consequential digital action has been made accountable: cryptographic signing, deterministic content addressing, third-party timestamping, public-key identity.

The openai-claw project is the first step in that direction. It is an open-source, MIT-licensed, npm-installable AI agent CLI in which every session is signed with a post-quantum key, hashed into a deterministic Merkle tree, and Bitcoin-anchored via standards-compliant OpenTimestamps proofs — and in which the entire chain of verification can be reproduced by a stranger in five commands, with no dependency on the agent itself.

We are publishing the protocol and the reference implementation. We expect — and welcome — independent implementations, audits, and proofs of vulnerability. The hardest problem in this space is not the cryptography. It is the social contract: convincing operators, regulators, and customers that "trust us" is no longer the right answer for autonomous AI work, and that the alternative is a small, boring, deterministic verification step that anyone can run.

If we are right about the trajectory, this will be the audit layer of the next decade.

Appendix A · Wire format reference (v1)

A.1 Leaf

```
{
  "v": 1,                              // schema version, integer
  "seq": 0,                            // monotonic 0..N-1 per session
  "ts": "2026-05-15T16:50:15.760Z",    // operator-supplied, untrusted
  "kind": "user_prompt"
         | "assistant_text"
```

```
        | "tool_call"
        | "tool_result"
        | "permission_decision"
        | "compaction"
        | "error",
  "payloadHash": "<hex sha256 of canonical-JSON(payload)>"
}
```

A.2 AttestationHeader

```
{
  "v": 1,
  "format": "openai-claw.attestation.v1",
  "sessionId": "...",
  "startedAt": "ISO-8601",
  "endedAt":   "ISO-8601",
  "leafCount": 6,
  "merkleRoot": "<hex sha256>",
  "suiteId": "ml-dsa-65",
  "publicKey":   "<base64 1952 bytes>",
  "publicKeyId": "<16-char base64url fingerprint>"
}
```

A.3 Sidecar

```
{
  "header":    AttestationHeader,
  "leaves":    Leaf[],
  "signature": "<base64 ~3309 bytes ML-DSA-65 signature over canonical-JSON(header)>",
  "anchor":    AnchorProof | undefined
}
```

A.4 AnchorProof

```
{
  "type": "opentimestamps-pending",
  "digest": "<hex sha256(canonical-JSON(header))>",
  "submittedAt": "ISO-8601",
  "calendars": [
    { "url": "https://alice.btc.calendar.opentimestamps.org",
      "ok":  true,
      "response": "<base64 calendar response bytes>" },
    ...
  ]
}
```

A.5 .ots file (per OpenTimestamps spec)

```
\x00OpenTimestamps\x00\x00Proof\x00          // 23 byte string identifier
```

```
\xbf\x89\xe2\xe8\x84\xe8\x92\x94             //  8 byte magic
\x01                                          //  1 byte version
\x08                                          //  1 byte op_sha256
<32 byte sha256(canonical-JSON(header))>     // 32 byte digest, raw
<calendar response bytes>                     // ops + pending attestation
```

Appendix B · Reproducing the audit demo

```
# 1. Clone the repo
git clone https://github.com/codenlighten/openai-claw
cd openai-claw/examples/audit-demo

# 2. Run the script
./verify.sh

# 3. Optional: install the standard OTS CLI for chain verification
pipx install opentimestamps-client
ots info proofs/sample-session.alice.ots
ots upgrade proofs/sample-session.alice.ots   # after ~3 hours
ots verify proofs/sample-session.alice.ots    # against the Bitcoin chain
```

Expected output (truncated):

```
== step 2: run the verifier against the committed fixtures
  format:            OK
  leafContinuity:    OK
  merkleRoot:        OK
  signature:         OK
  sessionAlignment:  OK
  anchorDigest:      OK
  publicKeyId:       TSR2Wtuvy-3Ls1Ov
  ...
  cryptographic audit: PASS

== step 3: inspect each .ots proof with the standard OpenTimestamps tool
  proofs/sample-session.alice.ots
    File sha256 hash: 545fde095bf6ee9a...
    ...
    verify PendingAttestation('https://alice.btc.calendar.opentimestamps.org')
```

Appendix C · References

1. **NIST FIPS 204** — Module-Lattice-Based Digital Signature Standard (ML-DSA), August 2024. https://csrc.nist.gov/pubs/fips/204/final
2. **CRYSTALS-Dilithium** — V. Lyubashevsky et al. "Dilithium: A Lattice-Based Digital Signature Scheme." *IACR Transactions on Cryptographic Hardware and Embedded Systems* (2018).
3. **RFC 8785** — A. Rundgren, B. Jordan, S. Erdtman. "JSON Canonicalization Scheme (JCS)." June 2020.

4. **OpenTimestamps Protocol** — P. Todd. https://github.com/opentimestamps/python-opentimestamps. Reference docs: https://opentimestamps.org
5. **Bitcoin** — S. Nakamoto. "Bitcoin: A Peer-to-Peer Electronic Cash System." October 2008.
6. **RFC 6962** — B. Laurie, A. Langley, E. Kasper. "Certificate Transparency." June 2013. (Related construction of a public transparency log via Merkle trees.)
7. **W3C Verifiable Credentials Data Model 2.0** — M. Sporny, D. Longley, D. Chadwick, eds. https://www.w3.org/TR/vc-data-model-2.0/
8. **`@smartledger/crypto`** — Lumen Crypto SDK with ML-DSA-44/65/87 implementations. https://www.npmjs.com/package/@smartledger/crypto
9. **`@smartledger/bsv`** — Bitcoin SV development framework with GDAF, LTP, StatusList2021. https://www.npmjs.com/package/@smartledger/bsv
10. **`@smartledger.technology/openai-claw`** — Reference implementation. https://www.npmjs.com/package/@smartledger.technology/openai-claw
11. **`@smartledger.technology/openai-claw-verify`** — Standalone verifier. https://www.npmjs.com/package/@smartledger.technology/openai-claw-verify
12. **Model Context Protocol** — Anthropic. "Security Best Practices." https://modelcontextprotocol.io/docs/tutorials/security/security_best_practices
13. **OWASP** — "MCP Security Cheat Sheet." OWASP Cheat Sheet Series. https://cheatsheetseries.owasp.org/cheatsheets/MCP_Security_Cheat_Sheet.html
14. **Black Hills Information Security** — "Model Context Protocol (MCP)." https://www.blackhillsinfosec.com/model-context-protocol/
15. **IT Pro** — "AI agents using Anthropic MCP could be a vector for supply chain attacks, claim researchers." https://www.itpro.com/security/ai-agents-using-anthropic-mcp-supply-chain-attacks-claim-researchers

---

© 2026 Gregory J. Ward, Bryan W. Daugherty, Shawn M. Ryan, and SmartLedger.Technology. This document is licensed under CC-BY-4.0. The reference implementations are MIT-licensed. Comments and corrections: greg@smartledger.technology.